

Supporting Document

TURO-ONBOARD: Product Overview & Design Document

Overview

TURO-ONBOARD is an AI-powered onboarding tool designed for platforms that help independent car rental operators move away from Turo's marketplace and build direct-booking businesses. The tool takes a Turo listing URL as input and generates a complete, personalized onboarding package in under 60 seconds pulling from real listing data rather than templates or averages.

The core insight behind it: the biggest drop-off in operator conversion isn't the product itself. It's the first call. Most operators don't switch because nobody has shown them, in their own numbers, what switching is actually worth. TURO-ONBOARD solves that.

The Problem

Turo charges operators 20–25% per booking in marketplace fees. An operator running 8 trips a month at an average of 3 days per trip is losing between \$900 and \$1,400 every month just in platform cuts. The economics of switching to direct bookings are clear but operators don't see them clearly, because no one has done the math for their specific vehicle and market.

On the sales side, onboarding teams face the same problem every time: they're walking into a call knowing the platform, but not knowing the operator. A generic pitch to a high-volume host in Rogers, Arkansas looks the same as a pitch to a new host in Miami, and operators notice.

The Solution

TURO-ONBOARD automates the research and preparation that should happen before every onboarding call. In one input: a Turo listing URL and it produces five things:

1. **Revenue Impact.** Calculates the operator's current effective daily rate after Turo's fees, recommends a competitive direct rate, and estimates the monthly revenue difference based on their actual trip history.
2. **Customer Persona.** Identifies who is most likely renting that specific vehicle in that market age range, use case, booking motivations, frustrations with Turo so the onboarding conversation can speak to the renter's experience, not just the platform's features.
3. **Website Copy.** Generates a hero headline, sub headline, three value propositions, and trust signals for the operator's direct booking site. All copy is specific to their vehicle and location.

4. **Onboarding Call Agenda.** Produces a 5-step call structure with key messages and suggested opening lines for each step. Whoever runs the call has a script that's tailored to that operator, not recycled from the last one.
5. **Features and Flags.** Recommends platform features that are most relevant given what the listing shows, and surfaces red flags low review counts, pricing inconsistencies, listing gaps that should be addressed before or during the call.

How It Works

The pipeline runs in three stages.

Stage 1: Scraping. A Playwright-powered headless browser loads the Turo listing, scrolls through the page to trigger lazy-loaded content, and extracts the HTML. A BeautifulSoup parser pulls structured data from JSON-LD schema blocks first which Turo embeds for SEO and falls back to DOM parsing and regex for anything not in the structured data. Output is a clean JSON object with vehicle specs, pricing, host information, location, features, and reviews.

Stage 2: AI Analysis. The listing JSON is sent to Cohere's Command model via the v2 Chat API. The prompt defines a strict JSON output schema and instructs the model to use specific, concrete language referencing actual numbers from the listing rather than generalizations. The model returns a complete onboarding package as structured JSON.

Stage 3: UI Rendering. A Streamlit app displays the package across five tabs, one per output section. All content is rendered as inline HTML to maintain consistent styling across Streamlit's rendering pipeline. Outputs can be exported as JSON or Markdown.

Technical Stack

- **Playwright:** headless browser automation for scraping Turo listings
- **BeautifulSoup + lxml:** HTML parsing and structured data extraction
- **Cohere Command (command-a-03-2025):** AI analysis and package generation via REST API
- **Streamlit:** web interface for input, output display, and export
- **Python / requests:** API communication and pipeline orchestration

Design Decisions Worth Noting

The scraper uses `headless=False` during development so the browser interaction is visible and easier to debug against Turo's bot detection. In production this would be flipped.

The Cohere call uses direct HTTP requests via the `requests` library rather than the SDK. This was a deliberate decision after encountering Windows-specific `async` conflicts between Streamlit's event loop and `httpx`, which the Cohere SDK uses internally.

All card components in the UI are rendered as single HTML blocks rather than using Streamlit's native components wrapped in injected HTML. Streamlit renders each `st.markdown()` call independently in its own DOM wrapper injected open and close div tags don't actually wrap the components between them, which causes empty visual boxes to appear. Single-block HTML eliminates this.

Current Limitations

Turo's listing pages are JavaScript-heavy and occasionally load inconsistently. The scraper includes scroll delays and a `h1` wait to handle this, but listings with aggressive bot detection can still fail. A retry mechanism and proxy rotation would improve reliability at scale. The AI output is only as good as the scraped data. For new listings with few reviews or minimal host information, the model infers and flags this but the package will naturally be less specific.

What's Next

The logical next step is a CRM integration pushing each generated package directly into the operator's record before the call is scheduled, so the sales team always walks in prepared. A batch processing mode for scraping and analyzing multiple listings in one run is also straightforward to add given the current pipeline structure. Longer term, tracking which package elements correlate with conversion would let the AI learn which revenue estimates, persona types, and call scripts actually close turning TURO-ONBOARD from a preparation tool into a feedback loop.